

Email Validation

Email validation is a process of verifying the authenticity and correctness of an email address to ensure it meets the required standards. This typically involves checking the format, domain, and other criteria defined by email protocols. Email validation is crucial in web applications to ensure accurate communication, prevent spam, and maintain data integrity. In the context of the provided Django project, the system validates a list of emails submitted via a web form, distinguishing between valid and invalid addresses. Valid emails are saved to both MySQL and Oracle databases, while invalid ones are identified and presented to the user.

- **ALL STEPS FOR CREATING EMAIL VALIDATION PROJECT.**

STEP 1: [Download Python Programming Language]

- This site <https://www.python.org/downloads/>

Python is a high-level, interpreted programming language known for its simplicity and readability. Created by Guido van Rossum, Python emphasizes code readability and ease of use, making it a popular choice for beginners and experienced developers alike. It supports multiple programming paradigms, including procedural, object-oriented, and functional programming. Python has a comprehensive standard library and a vast ecosystem of third-party packages, making it versatile for various applications, including web development, data science, artificial intelligence, automation, and more. Its syntax is clean and concise, promoting code readability and reducing the cost of program maintenance. Python's community-driven development and open-source nature contribute to its widespread adoption and continual evolution.

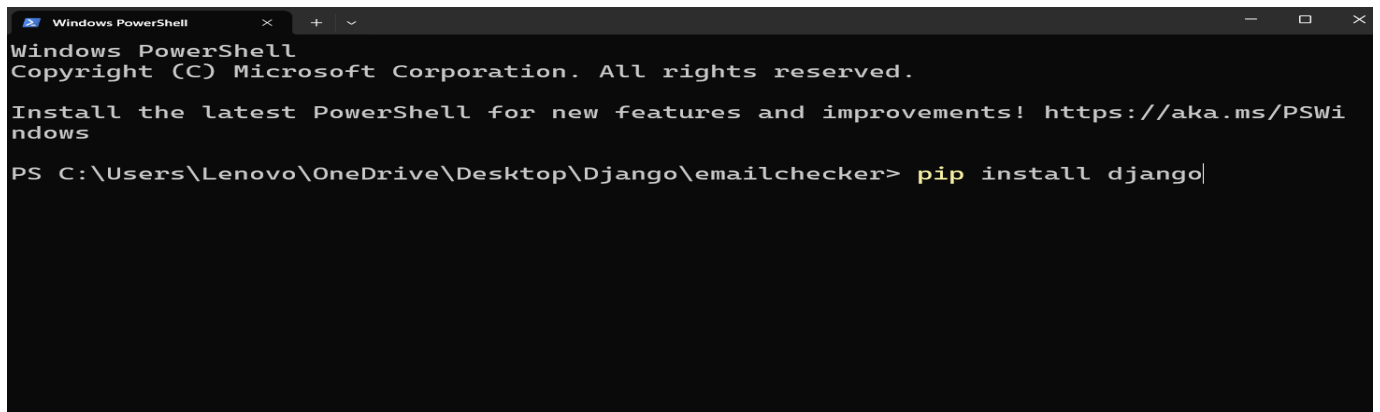
STEP 2: [Download Visual Studio Code]

- This site <https://code.visualstudio.com/download>

VSCode (Visual Studio Code) is a lightweight and versatile source code editor developed by Microsoft, available for download on Windows, Linux, and macOS. It provides a user-friendly interface, extensive language support, and a rich ecosystem of extensions for enhanced functionality. Known for its speed and responsiveness, VSCode is widely used by developers for various programming languages and offers features such as syntax highlighting, code completion, debugging, and Git integration.

STEP 3: [Install Django in Python]

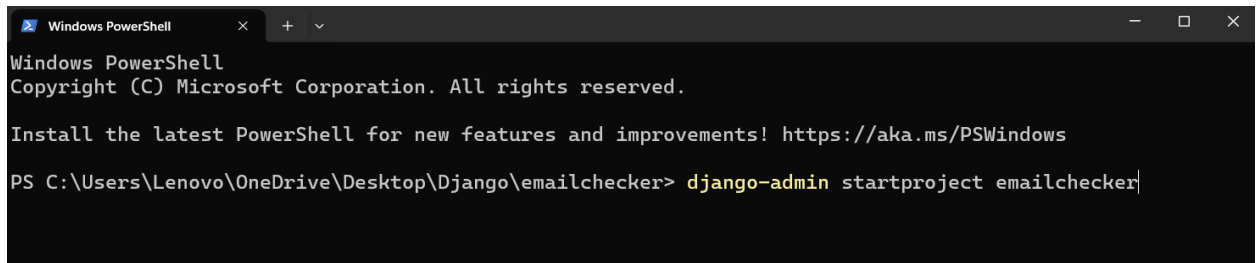
- **COMMAND:** `pip install django`

A screenshot of a Windows PowerShell terminal window. The title bar at the top says "Windows PowerShell" with standard window controls. The terminal text includes the Windows PowerShell copyright notice, a message to install the latest PowerShell, and a command prompt showing the user's path as "C:\Users\Lenovo\OneDrive\Desktop\Django\emailchecker" and the command "pip install django" being typed.

After installing Django using the **pip install django** command, you won't see any specific UI as it is a command-line operation. However, you can create a new Django project and run the development server to interact with the Django admin interface or any web application you build.

1] Create a Django Project:

- **COMMAND :** **django-admin startproject emailchecker**



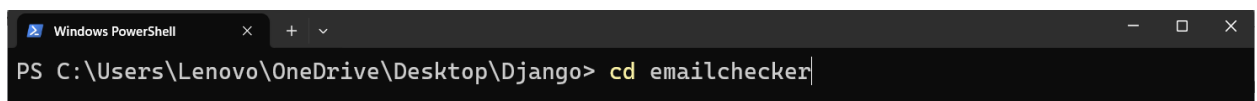
```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Lenovo\OneDrive\Desktop\Django\emailchecker> django-admin startproject emailchecker
```

2] Navigate to the Project Directory:

- **COMMAND :** **cd emailchecker**



```
Windows PowerShell

PS C:\Users\Lenovo\OneDrive\Desktop\Django> cd emailchecker
```

3] Django default files has created in folder:

- **manage.py:** This is a command-line utility that lets you interact with your project in various ways, such as running the development server and creating database tables.
- **emailchecker/ emailapp:** This directory contains the settings and configurations for your project. Key files within this directory include:
- **__init__.py:** An empty file that tells Python to treat the directory as a Python package.
- **settings.py:** Configuration settings for your Django project, including database settings, time zone, and more.
- **urls.py:** Defines the URL patterns for your project.
- **asgi.py and wsgi.py:** These files provide entry points for ASGI (Asynchronous Server Gateway Interface) and WSGI (Web Server Gateway Interface), which are specifications for communication between web servers and Django applications.
- **db.sqlite3:** The default SQLite database file. You might choose to use a different database by modifying the settings in settings.py.

INSTALL EXTENSIONS IN VSCODE:

1] Python IntelliSense: The Python IntelliSense extension for Visual Studio Code (VSCode) is a powerful tool that significantly enhances the Python development experience within the editor. It provides intelligent code completion, real-time suggestions, and documentation hints as developers write Python code. This extension leverages the power of the language server protocol (LSP) to analyze the Python codebase, offering accurate and context-aware suggestions for functions, variables, and modules. With features like auto-imports, parameter hints, and quick documentation lookup, Python IntelliSense streamlines coding workflows, helping developers write code faster and with fewer errors. This extension is an essential companion for Python developers using VSCode, providing a seamless and efficient coding environment.

2] DJANGO : Django Extension for Visual Studio Code is a plugin that enhances the development experience for Django projects within the VSCode editor. It provides features such as auto-completion for Django-specific syntax, easy navigation to template files, model fields, and views. Additionally, it supports running and debugging Django applications directly from VSCode, making it a valuable tool for Django developers by improving productivity and code quality in the integrated development environment.

3] HTML CSS Support: Visual Studio Code (VSCode) offers robust support for HTML and CSS development. The editor provides a rich set of features, including syntax highlighting, autocompletion, and code snippets for HTML and CSS. VSCode also supports Emmet abbreviations for faster HTML and CSS coding. The integrated IntelliSense provides context-aware suggestions, making it easier to write and navigate through code. Additionally, VSCode allows you to preview and debug web pages directly within the editor, enhancing the development workflow. The built-in Git integration further facilitates version control for web projects. Extensions like Live Server enable real-time previews of HTML and CSS changes in a web browser. Overall, VSCode provides a highly efficient and customizable environment for HTML and CSS development, making it a popular choice among web developers.

4] Django Templates: The Django Templates extension for Visual Studio Code (VSCode) enhances the development experience for Django web applications by providing advanced support for working with Django template files. It offers features such as syntax highlighting, auto-completion of Django template tags and filters, and intelligent code navigation. This extension streamlines the template editing process, making it easier for developers to write and maintain Django templates within the VSCode environment.

STEP 4: [Creating Templates folder in emailapp File]

- In Templates Folder we Have Created **email_check.html** file

CODE:

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Email Checker</title>
  <link rel="icon" type="image" href="Email_icon.ico">
  <style>
    body {
      background-color: silver;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
      margin: 0;
      background-image:
url(C:\Users\Lenovo\OneDrive\Desktop\Django\emailchecker\emailapp\templates\emailapp\Email_background.jpg);
      background-size: cover;
      background-position: center;
      background-repeat: no-repeat;
      margin: 0;
      height: 100vh;
    }

    .container {
      text-align: center;
      background-color: white;
      padding: 20px;
      border-radius: 10px;
      width: 80%;
      max-width: 600px;
      /* Increase the max-width for a larger container */
    }

    textarea#emails {
      margin-bottom: 10px;
      padding: 12px;
      /* Increase padding for a larger text area */
      width: 100%;
      box-sizing: border-box;
      font-size: 16px;
      /* Increase font size for better readability */
      resize: vertical;
      /* Allow vertical resizing */
    }

    button {
      background-color: steelblue;
      color: white;
```

```

padding: 10px;
border: none;
border-radius: 5px;
cursor: pointer;
margin: 5px;
/* Add margin for better spacing */
}

.success-message {
margin-top: 10px; /* Added margin for spacing */
color: green; /* Set color for valid emails */
}

.error-message {
margin-top: 10px; /* Added margin for spacing */
color: red; /* Set color for invalid emails */
}

/* Updated styling to place valid_emails on the left and invalid_emails on the right */
#valid_output {
float: left;
width: 48%; /* Adjust width for better alignment */
}

#invalid_output {
float: right;
width: 48%; /* Adjust width for better alignment */
}

.copy-button {
background-color: Silver;
color: white;
padding: 5px;
text-align: center;
text-decoration: none;
display: inline-block;
font-size: 12px;
border-radius: 5px;
cursor: pointer;
margin-top: 10px; /* Added margin for spacing */
}

/* Added style for the "Copy Valid Emails" button */
#copy_valid_button {
background-color: Silver;
color: white;
padding: 5px;
text-align: center;
text-decoration: none;
display: inline-block;
font-size: 12px;
border-radius: 5px;
cursor: pointer;
margin-top: 10px; /* Added margin for spacing */
}

/* Added style for the "Reset" button */
.reset-button {
background-color: gray;
color: white;
padding: 10px;

```

```

border: none;
border-radius: 5px;
cursor: pointer;
margin: 5px;
/* Add margin for better spacing */
}
textarea#emails {
margin-bottom: 10px;
padding: 12px;
width: 100%;
box-sizing: border-box;
font-size: 16px;
resize: vertical;
overflow-y: scroll; /* Added scroll bar for vertical overflow */
}
textarea#emails {
margin-bottom: 10px;
padding: 12px;
width: 100%;
box-sizing: border-box;
font-size: 16px;
resize: vertical;
overflow-y: scroll; /* Added scroll bar for vertical overflow */
}
</style>
</head>
<body>
<div class="container">
<h1 style="color: steelblue;">Email Validation</h1>

<form method="post">
  {% csrf_token %}

  <label for="emails">Enter Emails:</label>
  <textarea id="emails" name="emails" required rows="4"></textarea>

  <div>
    <button type="submit">Process</button>
    <button type="button" class="reset-button" onclick="location.reload()">Reset</button>
  </div>

  <div>
    <div id="invalid_output">
      {% if invalid_emails %}
        <p class="error-message">Invalid emails</p>
        <ul>
          {% for email in invalid_emails %}
            <li>{{ email }}</li>
          {% endfor %}
        </ul>

        <button class="copy-button" onclick="copyToClipboard('invalid_output')">Copy Invalid Emails</button>

        <button type="button" class="reset-button" onclick="resetSection('invalid_output')">Reset</button>
      {% endif %}
    </div>

    <div id="valid_output">
      {% if valid_emails %}
        <p class="success-message">Valid emails</p>
        <ul>

```

```

        {% for email in valid_emails %}
            <li>{{ email }}</li>
        {% endfor %}
    </ul>

    <button id="copy_valid_button" onclick="copyToClipboard('valid_output')>Copy Valid Emails</button>

    <button type="button" class="reset-button" onclick="resetSection('valid_output')>Reset</button>
{% endif %}
</div>
</div>
</form>

<script>
function copyToClipboard(elementId) {
    var copyList = document.getElementById(elementId).querySelector('ul');
    var textContent = Array.from(copyList.children).map(li => li.textContent).join('\n');
    var tempTextArea = document.createElement('textarea');
    tempTextArea.value = textContent;
    document.body.appendChild(tempTextArea);
    tempTextArea.select();
    document.execCommand('copy');
    document.body.removeChild(tempTextArea);
    alert("Copied to clipboard:\n" + textContent);
}

function resetSection(elementId) {
    document.getElementById(elementId).innerHTML = "";
}
</script>
</div>
</body>
</html>

```

- I Have created Views.py file Under
Location:C:\Users\Lenovo\OneDrive\Desktop\Django\email checker\email app

Code:

```

from django.core.validators import validate_email
from django.core.exceptions import ValidationError
from django.shortcuts import render
from .models import ValidEmail

```

```

def email_check(request):

```

```

    valid_emails = []
    invalid_emails = []

```

```

    if request.method == 'POST':

```



```

emails = request.POST.get('emails', "").split()

for email in emails:
    try:
        validate_email(email.strip())
        valid_emails.append(email)

        existing_entry = ValidEmail.objects.filter(email=email.strip()).first()

        if not existing_entry:
            # Save to MySQL
            ValidEmail.objects.save_to_mysql(email=email.strip())

            # Save to Oracle
            ValidEmail.objects.save_to_oracle(email=email.strip())
    except ValidationError:
        invalid_emails.append(email)

return render(request, 'emailapp/email_check.html', {'valid_emails':
valid_emails, 'invalid_emails': invalid_emails})

```

• **Also ADD Code in models.py file under this**
Location C:\Users\Lenovo\OneDrive\Desktop\Django\emailchecker\emailapp

CODE:

```

from django.db import models

class ValidEmailManager(models.Manager):
    def save_to_mysql(self, email):
        return self.create(email=email)

    def save_to_oracle(self, email):
        return self.using('oracle_db').create(email=email)

```

```

def get_from_mysql(self):
    return self.all()

def get_from_oracle(self):
    return self.using('oracle_db').all()

class ValidEmail(models.Model):
    email = models.EmailField(unique=True)
    submission_date = models.DateField(auto_now_add=True)

    objects = ValidEmailManager()

def __str__(self):
    return self.email

```

STEPS 5: [In Settings.py Edit Some Code]

- **FIRST: [INSTALLED_APPS]**

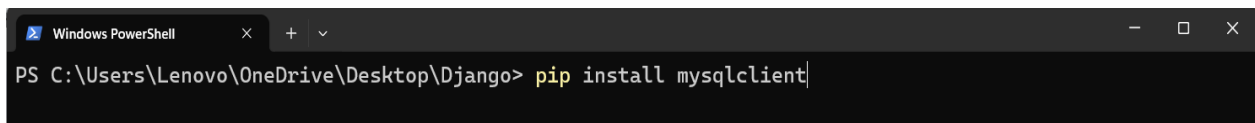
```

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'emailapp', <-----This added
]

```

→ **We want install mysqlclient in python**

COMMAND: pip install mysqlclient



```

Windows PowerShell
PS C:\Users\Lenovo\OneDrive\Desktop\Django> pip install mysqlclient

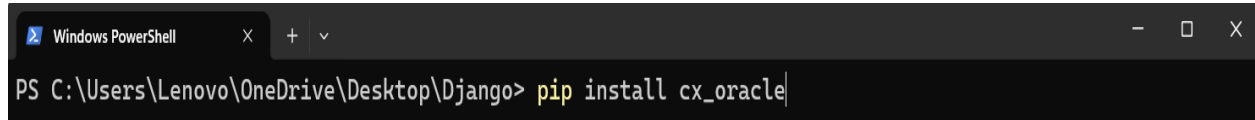
```

USECASE :

The `mysqlclient` is a Python library that serves as a MySQL database adapter, allowing Python applications to interact with MySQL databases seamlessly. It is commonly used as a MySQL database connector in Django projects. By providing a Pythonic interface to MySQL databases, `mysqlclient` enables developers to execute SQL queries, manage database connections, and handle data transactions efficiently. It is known for its reliability, performance, and compatibility with the Django ORM (Object-Relational Mapping) system, making it a popular choice for integrating MySQL databases with Python applications. `mysqlclient` enhances the development experience by facilitating smooth communication between Python code and MySQL databases, contributing to robust and scalable web application development.

→ Install also `cx_oracle` in python

- **COMMAND:** `pip install cx_oracle`



```
Windows PowerShell
PS C:\Users\Lenovo\OneDrive\Desktop\Django> pip install cx_oracle
```

USECASE:

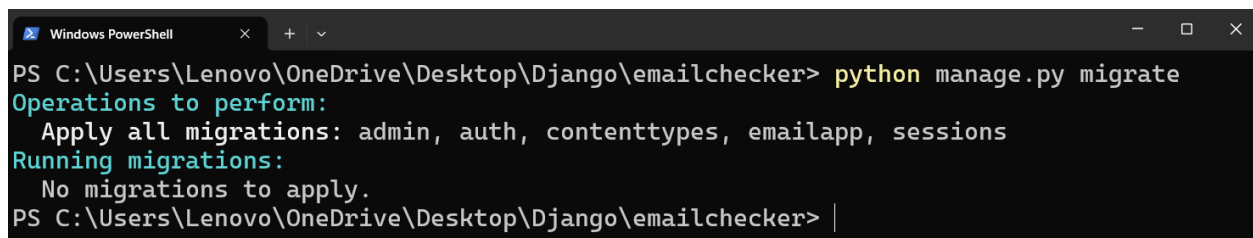
`cx\_Oracle` is a Python extension module that serves as an interface to the Oracle Database. It enables seamless connectivity between Python applications and Oracle databases, allowing developers to execute SQL and PL/SQL statements, fetch results, and manage database transactions with ease. `cx\_Oracle` supports Oracle's advanced features, making it suitable for a wide range of applications, from simple scripts to complex database-driven web applications. Known for its performance and reliability, this module provides efficient data access and manipulation, making it a preferred choice for integrating Python applications with Oracle Database systems. Its usage extends across diverse domains, including enterprise-level applications, data analytics, and business intelligence, contributing to the robustness and scalability of Oracle-backed Python projects.

**SECOND:[DATABASES]-->Present in Settings.py Under this
Location C:\Users\Lenovo\OneDrive\Desktop\Django\emailchecker\emailchecker**

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.mysql',  
        'NAME': 'django_project',  
        'USER': 'Username',  
        'PASSWORD': 'Password',  
        'HOST': '127.0.0.1',  
        'PORT': '3306',  
    },  
    'oracle_db':{  
  
        'ENGINE': 'django.db.backends.oracle',  
        'NAME': 'localhost:1521/x',  
        'USER': 'Username',  
        'PASSWORD': 'Password',  
    },  
}
```

STEP 7: [Now, We want to run migrate command in terminal]

- **COMMAND :** python manage.py migrate



```
Windows PowerShell  
PS C:\Users\Lenovo\OneDrive\Desktop\Django\emailchecker> python manage.py migrate  
Operations to perform:  
  Apply all migrations: admin, auth, contenttypes, emailapp, sessions  
Running migrations:  
  No migrations to apply.  
PS C:\Users\Lenovo\OneDrive\Desktop\Django\emailchecker> |
```

USECASE:

The `python manage.py migrate` command in Django plays a crucial role in managing database schema changes throughout the development lifecycle. It is a fundamental aspect of Django's migration system, which facilitates the synchronization of the database schema with the current state of the application's models. When developers make changes to the models, such as adding or modifying fields, the `migrate` command analyzes the differences between models and the existing database schema, generating and applying the necessary SQL statements to reflect these changes.

[Run makemigrations after Migrate Command]

- **COMMAND :** python manage.py makemigrations

```
PS C:\Users\Lenovo\OneDrive\Desktop\Django\emailchecker> python manage.py makemigrations
No changes detected
```

USECASE:

The `python manage.py makemigrations` command in Django is a crucial step in the database migration process. This command is used to generate new database migration files based on changes detected in the project's models. When developers modify their Django models, such as adding new fields, changing existing ones, or creating new models altogether, the `makemigrations` command analyzes these alterations and creates migration files that capture the changes in a format Django understands. These migration files, typically found in the `migrations` directory of each app, serve as a blueprint for updating the database schema to match the updated model definitions. Each migration file contains Python code that instructs Django on how to apply or revert the changes to the database. By running `makemigrations`, developers ensure that their database schema stays synchronized with the evolving structure of their Django models. This command plays a pivotal role in maintaining the integrity and consistency of the database throughout the development lifecycle.

STEP 8: [Runserver Command]

COMMAND: python manage.py runserver

```
Windows PowerShell
PS C:\Users\Lenovo\OneDrive\Desktop\Django\emailchecker> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

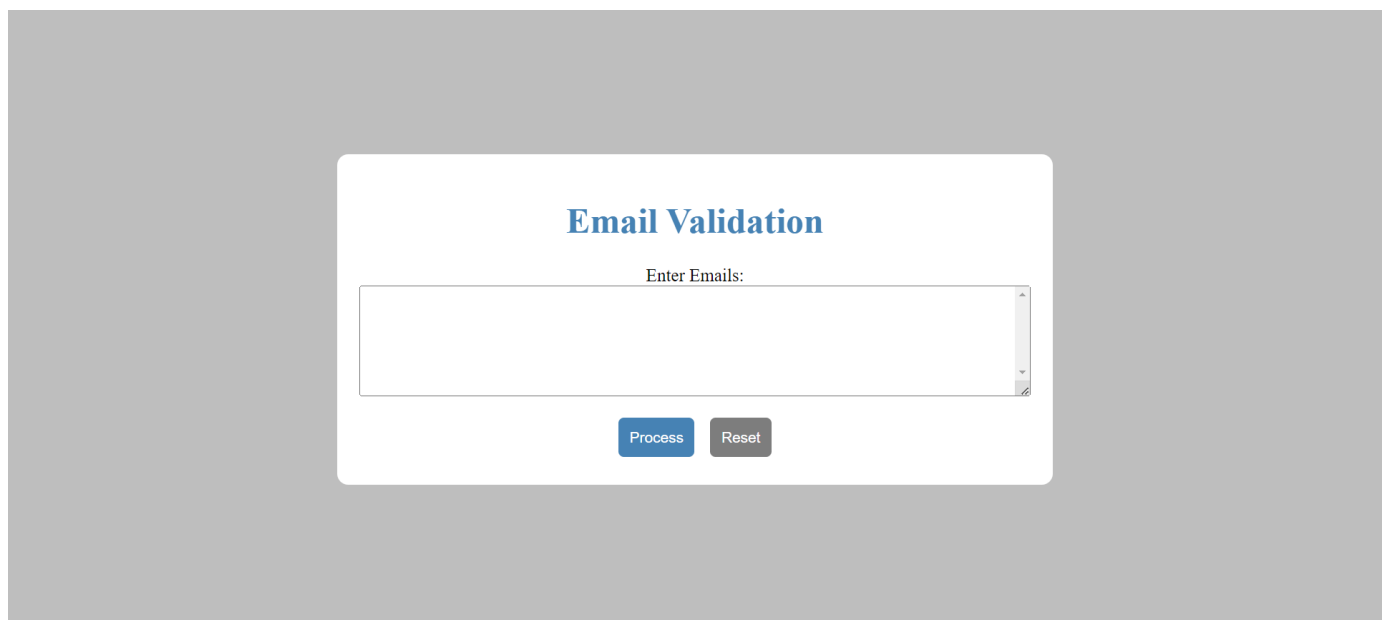
System check identified no issues (0 silenced).
February 28, 2024 - 19:47:01
Django version 5.0.1, using settings 'emailchecker.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

USECASE:

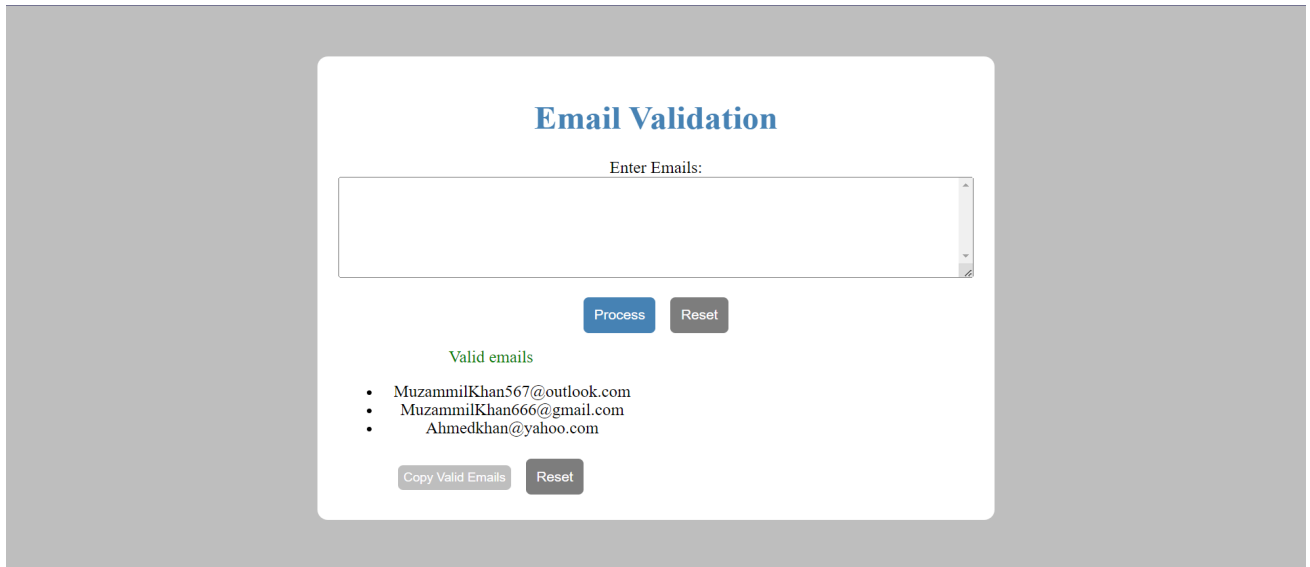
In the email checker Django project, the `python manage.py runserver` command plays a pivotal role in the development and testing phases. When executed, this command starts the Django development server, allowing developers to locally preview and interact with the email checker application. The development server listens for incoming HTTP requests on a specified port, typically 8000 by default, enabling seamless testing of the email verification functionality.

Using this command facilitates a rapid development cycle, as developers can make changes to the codebase and immediately observe the impact by accessing the local server through a web browser. The `runserver` command also provides real-time feedback in the terminal, displaying HTTP request logs and potential errors. This quick and efficient development server is crucial for iterative development, enabling developers to fine-tune the email checking logic, validate user inputs, and identify and address any issues promptly. While it is not suitable for production use, the `runserver` command remains an indispensable tool in the Django developer's toolkit, streamlining the process of building and refining the email checker application.

- Email Checker Website Occur: **<http://127.0.0.1:8000/>**



STEP 9 :[User enter valid email address in enter emails text box]



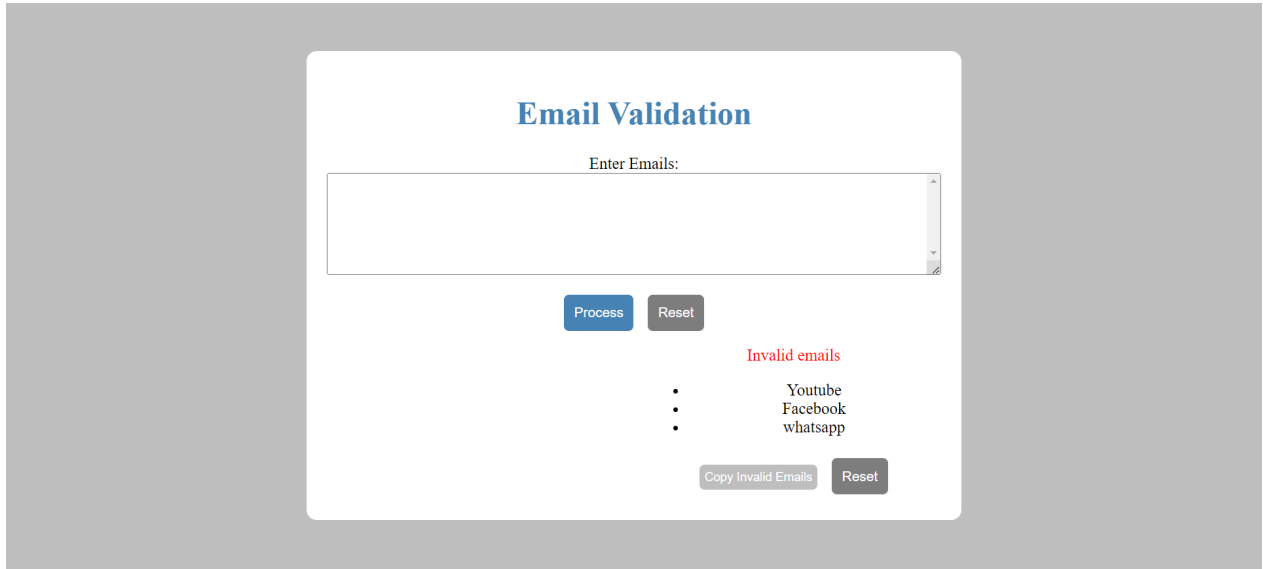
The screenshot displays a web application titled "Email Validation". At the top, there is a label "Enter Emails:" above a large, empty text input box. Below the input box are two buttons: "Process" (in blue) and "Reset" (in grey). Underneath these buttons, the text "Valid emails" is shown in green. Below this, a bulleted list of three email addresses is displayed: "MuzammilKhan567@outlook.com", "MuzammilKhan666@gmail.com", and "Ahmedkhan@yahoo.com". At the bottom of the list, there are two buttons: "Copy Valid Emails" (in grey) and "Reset" (in grey).

In scenarios where users input multiple email addresses in a designated text box, implementing a system to categorize and validate these addresses can significantly enhance the user experience. Upon submission, the application can parse and validate each entered email address, segregating them into two sections: valid and invalid. In the valid section, only those email addresses that meet the required format and structure criteria are displayed. This provides users with immediate feedback on the accuracy of their input.

To further enhance usability, developers can include features such as the ability to copy valid email addresses directly from the valid section. This functionality streamlines the process for users, allowing them to easily extract and use the correct email addresses without the need for manual copying or retyping. This can be particularly valuable in scenarios where users need to share or utilize the validated email addresses in subsequent actions, such as sending invitations, notifications, or messages.

By implementing this approach, developers not only ensure data accuracy but also contribute to an efficient and user-friendly experience, allowing users to seamlessly work with valid email addresses while minimizing the risk of errors or miscommunication.

STEP 10:[User enter Invalid email address in enter emails text box]



The screenshot displays a web application titled "Email Validation". It features a large text input field labeled "Enter Emails:". Below the input field are two buttons: "Process" (in blue) and "Reset" (in grey). Underneath these buttons, the text "Invalid emails" is shown in red. Below this text is a bulleted list of invalid email addresses: "Youtube", "Facebook", and "whatsapp". At the bottom of the interface, there are two more buttons: "Copy Invalid Emails" (in grey) and "Reset" (in grey).

In situations where users input multiple email addresses into a designated text box, implementing a robust system to categorize and validate these addresses becomes essential for an enhanced user experience. Upon submission, the application can intelligently parse and validate each entered email address, segregating them into two distinct sections: valid and invalid.

In the invalid section, users can quickly identify and review email addresses that do not meet the required format and structure criteria. This immediate feedback not only aids users in recognizing and rectifying errors but also contributes to a more transparent and user-friendly interface.

To further enhance usability, developers can incorporate features enabling users to copy invalid email addresses directly from the invalid section. This functionality streamlines the correction process, allowing users to easily extract and address the problematic email addresses without manual intervention. This proves especially beneficial when users need to rectify or communicate issues with the provided email addresses.

By implementing such a comprehensive approach, developers ensure not only data accuracy but also contribute significantly to an efficient and user-friendly experience. Users can seamlessly work with both valid and invalid email addresses, minimizing the risk of errors or miscommunication and promoting a smoother interaction within the application. This validation and categorization mechanism provides users with valuable insights and tools

STEP 11:[All valid emails addresses store in MYSQL Databases]

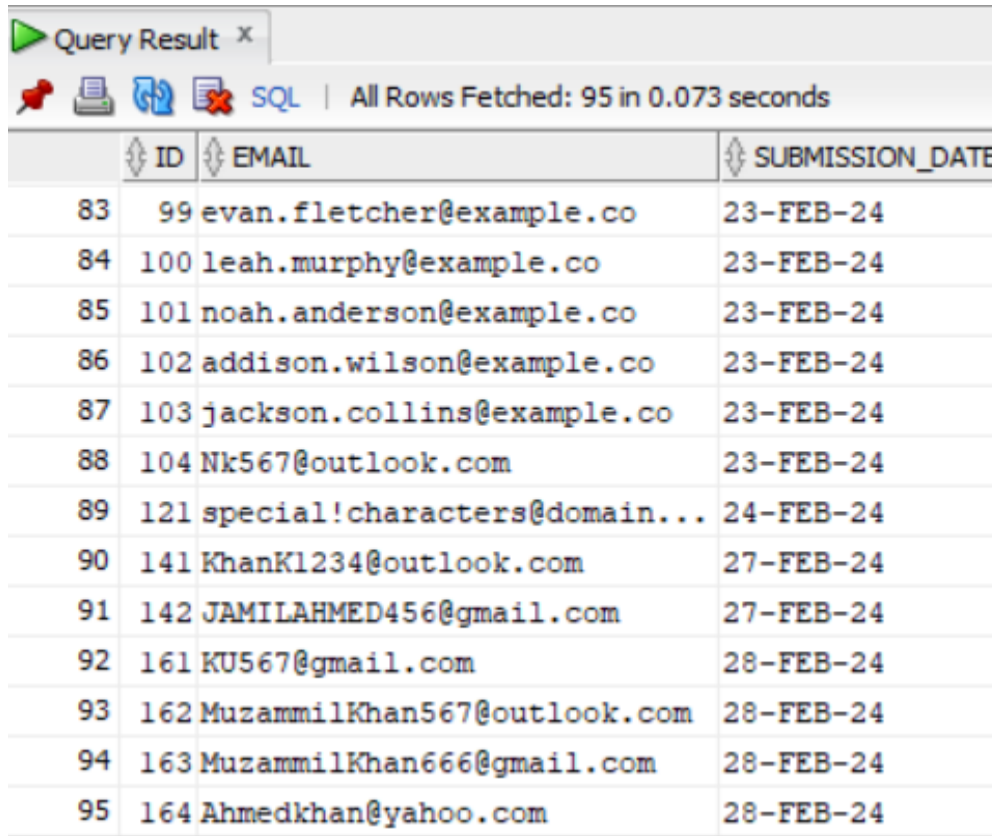
MYSQL WorkBench:

	id	email	submission_date
	393	noah.anderson@example.com	2024-02-23
	394	addison.wilson@example.com	2024-02-23
	395	jackson.collins@example.com	2024-02-23
	396	chloe.robertson@example.com	2024-02-23
	397	evan.fletcher@example.co	2024-02-23
	398	leah.murphy@example.co	2024-02-23
	399	noah.anderson@example.co	2024-02-23
	400	addison.wilson@example.co	2024-02-23
	401	jackson.collins@example.co	2024-02-23
	402	Nk567@outlook.com	2024-02-23
	403	special!characters@domain.com	2024-02-24
	404	KhanK1234@outlook.com	2024-02-27
	405	JAMILAHMED456@gmail.com	2024-02-27
	406	KU567@gmail.com	2024-02-28
	407	MuzammilKhan567@outlook.com	2024-02-28
	408	MuzammilKhan666@gmail.com	2024-02-28
	409	Ahmedkhan@yahoo.com	2024-02-28
+	NULL	NULL	NULL

In a system where user-entered email addresses undergo validation and are deemed valid, the application can intelligently store these validated email addresses along with the current date in a MySQL database. This process involves capturing the accurate email entries that meet the required format and structure criteria, and associating them with the timestamp of their validation. Storing this information in MySQL not only ensures a reliable record of valid email addresses but also allows for easy retrieval and analysis based on submission dates. This database integration enhances the application's functionality by providing a structured and organized repository of validated emails, facilitating various operations such as user management, communication history, and data analysis. Overall, this approach contributes to the efficiency and data integrity of the system, allowing for seamless interaction with and utilization of validated email addresses in future application processes.

STEP 12:[All valid emails addresses Also store in Oracle Databases]

Oracle sql developer:



	ID	EMAIL	SUBMISSION_DATE
83	99	evan.fletcher@example.co	23-FEB-24
84	100	leah.murphy@example.co	23-FEB-24
85	101	noah.anderson@example.co	23-FEB-24
86	102	addison.wilson@example.co	23-FEB-24
87	103	jackson.collins@example.co	23-FEB-24
88	104	Nk567@outlook.com	23-FEB-24
89	121	special!characters@domain...	24-FEB-24
90	141	KhanK1234@outlook.com	27-FEB-24
91	142	JAMILAHMED456@gmail.com	27-FEB-24
92	161	KU567@gmail.com	28-FEB-24
93	162	MuzammilKhan567@outlook.com	28-FEB-24
94	163	MuzammilKhan666@gmail.com	28-FEB-24
95	164	Ahmedkhan@yahoo.com	28-FEB-24

- Storing all valid emails in Oracle SQL Developer with the current date involves capturing and persisting accurate email data for future reference. As users input multiple email addresses, the application validates and categorizes them, retaining only those that adhere to the required format and structure. Once validated, these email addresses, along with their submission date, are stored in an Oracle database using SQL Developer. The inclusion of the submission date allows for a chronological record, providing insights into when each email was added to the system. This database storage approach not only ensures data accuracy but also establishes a comprehensive historical log of valid emails, facilitating future analysis, tracking, and management. It forms a crucial part of a robust system that allows for the secure and organized storage of validated email information in an Oracle database, contributing to effective data management and retrieval within the application.

- In conclusion, the development of the Email Validation project involves a systematic process aimed at creating a user-friendly and efficient web application. The project starts with the setup of the Python programming language and Visual Studio Code, followed by the installation of Django, a powerful web framework for Python. Key steps include creating a Django project, configuring the database settings for MySQL and Oracle, and integrating essential extensions in Visual Studio Code for enhanced development.
- The core functionality revolves around email validation, ensuring the authenticity and correctness of user-provided email addresses. The Django application categorizes entered emails into valid and invalid sections, providing users with immediate feedback. The ability to copy valid or invalid emails streamlines user interactions and improves overall usability.
- MySQL and Oracle databases are utilized to store valid emails along with their submission dates, contributing to data integrity and historical tracking. The project showcases the versatility of Django in managing multiple database connections seamlessly.
- The comprehensive use case scenarios illustrate the application's behavior when users input both valid and invalid email addresses, showcasing the system's ability to handle and categorize emails effectively.
- Ultimately, the Email Validation project demonstrates best practices in Django development, database management, and user interface design, contributing to a robust and reliable web application that addresses the crucial need for accurate and secure email validation.